Nutrify: Desi Dish Recognition and Calorie Estimation

Deliverable 2 – Model Architecture and Empirical Evaluation

Muhammad Huzaifa (24L-3019)
Muhammad Abdullah Rasheed (24L-3002)
Artificial Intelligence, BSE-5B, FAST-NUCES Lahore

November 2026

1 Phase 1: Problem Definition

1.1 Problem Statement

Tracking what we eat is one of the simplest ways to manage weight and health, but it only works if logging a meal is quick. In South Asia it usually is not. Popular nutrition apps are built around Western food databases, so everyday dishes such as biryani, nihari, haleem and karahi are either missing, listed under unfamiliar names, or represented by a generic entry that does not match what is on the plate.

Users are left to search manually, guess which entry is closest, and estimate the portion by eye. The error is large because the calorie content of desi dishes depends heavily on the oil or ghee used, the cut of meat and the serving size, none of which a generic entry captures. Most people give up after a few days, which defeats the purpose of tracking.

Nutrify addresses this by turning a single photo of a meal into a dish name and a calorie range. The system has three parts: an image classification model that recognises 14 popular desi dishes, a calorie module that maps the recognised dish to a per-serving value with an adjustable portion size, and a web application with image upload and camera capture that returns the dish name, a confidence score and a calorie range in seconds.

1.2 Target Audience

Primary users. Everyday people in Pakistan and the wider South Asian region who want to keep an eye on what they eat, for example to lose or gain weight, follow a fitness plan, or simply eat more mindfully. They are comfortable taking a photo on a phone but do not want to type in meals or look up nutrition tables.

Secondary users. Students, fitness coaches and dietitians who need a quick estimate when discussing local food, and who currently rely on printed tables or Western-centric apps.

Research gap. Widely used food recognition benchmarks such as Food-101 are dominated by Western dishes, and there are few public, labelled image collections for Pakistani food. As a result, models trained on those benchmarks do not transfer well to dishes like nihari or haleem. This project helps fill that gap by assembling a curated dataset of desi dishes, documenting how it was collected and cleaned, and evaluating how the model behaves on look-alike dishes, different lighting and restaurant versus home-cooked photos.

1.3 AI Justification

Why not rule-based programming. Recognising a dish from a photo cannot be written as a set of if–else rules. The same dish looks different from one cook, restaurant or camera to the next, and the lighting, plating and background change from photo to photo. Dishes such as

nihari, haleem and aloo gosht are also visually similar brown stews, so simple colour or texture rules would confuse them. Any hand-written rule set would need constant exceptions and would still fail on photos it has not seen.

Why deep learning fits. Convolutional neural networks learn the relevant visual features, from edges and textures up to whole-dish patterns, directly from labelled examples instead of relying on features chosen by hand. This is the standard approach for image classification and it handles the variation in appearance that defeats fixed rules.

Why transfer learning. Collecting thousands of images per dish is not realistic for a student project. Starting from a network pretrained on a large general image collection (EfficientNet-B0) lets us reuse features it has already learned and fine-tune it on roughly 150 to 300 images per dish. We will compare it against a baseline CNN trained from scratch to show the benefit. EfficientNet-B0 is also small enough to serve through a web application.

Where AI stops. Machine learning is used only for the part that needs it, recognising the dish. The calorie value comes from a verified per-serving entry in a food composition database, scaled by the chosen portion size. Keeping that step deterministic makes the numbers traceable and easier to defend than a model that guesses calories directly.

2 Phase 2: Data Acquisition and Preprocessing

2.1 Collection

Raw images were collected from two sources, giving 6,193 images across the 14 dishes before any cleaning. Kaggle contributes 1,154 of them and Bing scraping the remaining 5,039.

Bing image scraping. The script `collect_images.py` downloads images from Bing using the `icrawler` package, with five query variants per dish (for example the plain dish name, "Pakistani", "homemade" and "restaurant plate"). Every downloaded URL is logged in `raw_metadata.csv`, so each image can be traced back to where it came from.

Kaggle dataset. The script `import_dataset.py` imports four classes from the Kaggle "Indian Food Classification" dataset (Pushkar Jain, Kaggle user `l33tc0d3r`) and maps them onto our dishes: `samosa` → samosa (262 images), `pakode` → pakora (278), `dal_makhani` → daal (321) and `kaathi_rolls` → paratha_roll (293). The remaining ten dishes come only from Bing.

Top-up scraping. After the first cleaning run, three dishes were below or close to our target of 150 images: chicken_karahi (128 images), aloo_gosht (167) and halwa_puri (171). We re-ran `collect_images.py` for these three dishes only, using its `--dishes` and `--set extra` options, which apply eight different search phrasings (for example "recipe", "dhaba style", "Lahore" and "home cooked") with up to 60 results each. This added 1,180 raw images (chicken_karahi 410, aloo_gosht 382, halwa_puri 388). Many of the new downloads duplicated images we already had, so cleaning removed a larger share from these three dishes (see Table 3). The cleaning and preprocessing scripts were then re-run on the combined collection, and all numbers in this report refer to that final run.

Dish	Raw images	Source
halwa_puri	702	Bing
chicken_karahi	639	Bing
daal	636	Bing + Kaggle (dal makhani, 321)
aloo_gosht	624	Bing
paratha_roll	566	Bing + Kaggle (kathi rolls, 293)
samosa	556	Bing + Kaggle (samosa, 262)
pakora	525	Bing + Kaggle (pakode, 278)
seekh_kebab	317	Bing
chicken_biryani	296	Bing
nihari	291	Bing
chicken_tikka	289	Bing
pulao	261	Bing
haleem	254	Bing
palak_paneer	237	Bing
Total	**6,193**	

Table 1: Raw images per dish and their source, before cleaning (including the top-up scraping).

2.2 Cleaning Implementation

Cleaning is done by `clean_data.py`, which never deletes the raw files: the images that pass are copied into a separate `data/clean` folder, together with `clean_metadata.csv` and `cleaning_report.csv`. The script applies the following checks:

- **Missing labels and duplicate metadata rows:** rows with no dish label, or repeated rows, are dropped.
- **Missing or corrupt files:** images that are not on disk or cannot be opened are dropped.
- **Tiny images:** images whose shorter side is below 150 pixels are dropped.
- **Extreme aspect ratio:** images with an aspect ratio above 3 are dropped.
- **Blur:** images whose Laplacian variance is below 40 are dropped as blurry.
- **Exact duplicates:** images with an identical MD5 hash are dropped.
- **Near duplicates:** images whose perceptual hash distance to an earlier image is at most 4 are dropped.

The script also records the width, height, brightness and blur score of every kept image, which are used for the exploratory analysis. Of the 6,193 raw images, 1,926 (31.1%) were removed and 4,267 (68.9%) were kept. Table 2 shows what each step removed, as reported in `cleaning_report.csv`; the report does not list separate counts for the missing-file, corrupt-file and tiny-image checks.

Cleaning step	Images removed
Missing label	0
Duplicate metadata rows	0
Exact duplicates (MD5)	1,390
Blurry (Laplacian variance < 40)	253
Near duplicates (perceptual hash ≤ 4)	278
Extreme aspect ratio (> 3)	5
Total removed	**1,926**
Images kept	**4,267**

Table 2: Images removed by each cleaning step (from `cleaning_report.csv`).

Table 3 gives the effect of cleaning on each dish. We also checked a random sample of 24 cleaned images from each of the three top-up dishes by eye. The dish was almost always correct and the images were sharp, but text and watermarks were common: about 12 of 24 sampled aloo_gosht images and about 12 of 24 halwa_puri images contained a text banner, a video-thumbnail graphic or a stock-photo watermark, against about 5 of 24 for chicken_karahi. This is a small sample, so these rates are only rough (see the limitations below).

Dish	Raw	Kept after cleaning
samosa	556	498
pakora	525	482
daal	636	461
paratha_roll	566	422
chicken_karahi	639	295
aloo_gosht	624	286
halwa_puri	702	263
chicken_biryani	296	248
seekh_kebab	317	241
nihari	291	236
haleem	254	227
chicken_tikka	289	224
pulao	261	197
palak_paneer	237	187
Total	**6,193**	**4,267**

Table 3: Images per dish before and after cleaning.

2.3 Feature Strategy

The script `preprocess.py` prepares the cleaned images for model training in the following way.

Label encoding. The 14 dish names are sorted alphabetically and mapped to integer ids: aloo_gosht = 0, chicken_biryani = 1, chicken_karahi = 2, chicken_tikka = 3, daal = 4, haleem = 5, halwa_puri = 6, nihari = 7, pakora = 8, palak_paneer = 9, paratha_roll = 10, pulao = 11, samosa = 12 and seekh_kebab = 13. The mapping is saved in `classes.json`.

Resizing and scaling. Every image is resized to 224×224 pixels, the input size of EfficientNet-B0, and its pixel values are scaled to the range $[0, 1]$.

Stratified split. The 4,267 cleaned images are divided 70/15/15 into training, validation and test sets. The split is stratified by dish, so each set keeps roughly the same class proportions,

and it uses a fixed random seed (42) so it can be reproduced. Because duplicates were removed before splitting, near-identical images cannot end up in both the training and test sets.

Split	Images
Train	2,986
Validation	640
Test	641
Total	**4,267**

Table 4: Size of each split.

Normalisation. The per-channel mean and standard deviation are computed from the training set only, so no information from the validation or test images leaks into preprocessing. They are saved in `norm_stats.json` and will be used to normalise the inputs during model training.

Channel	Mean	Std
Red	0.5855	0.2877
Green	0.4875	0.2854
Blue	0.3670	0.3069

Table 5: Channel statistics of the training set (pixel values in $[0, 1]$).

2.4 Exploratory Data Analysis

The charts below were generated by `eda.py` from the cleaned and raw metadata.

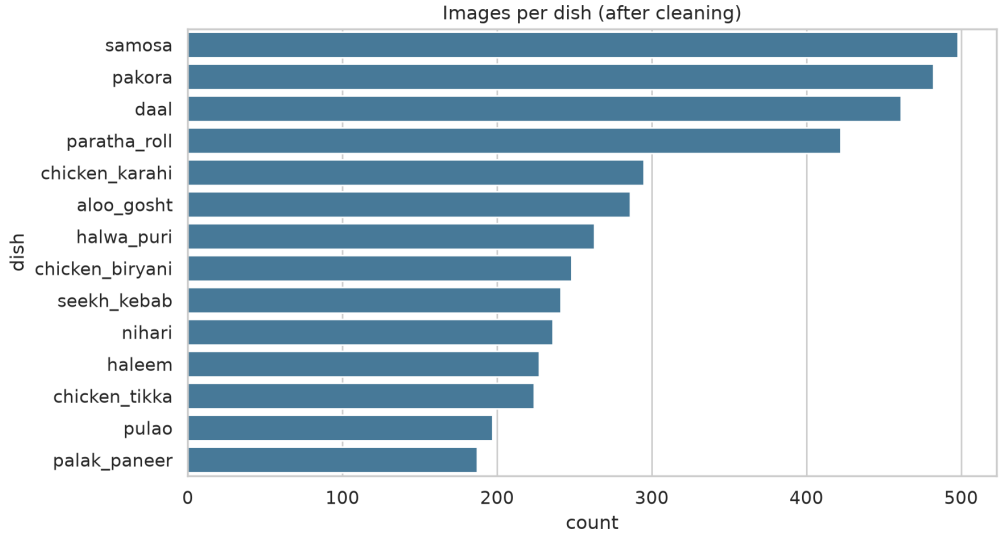


Figure 1: Images per dish after cleaning.

The classes are imbalanced. Samosa has the most images (498) and palak_paneer the fewest (187), a ratio of about 2.7 to 1. The four largest classes (samosa, pakora, daal and paratha_roll) are exactly the four dishes that also draw on the Kaggle dataset.

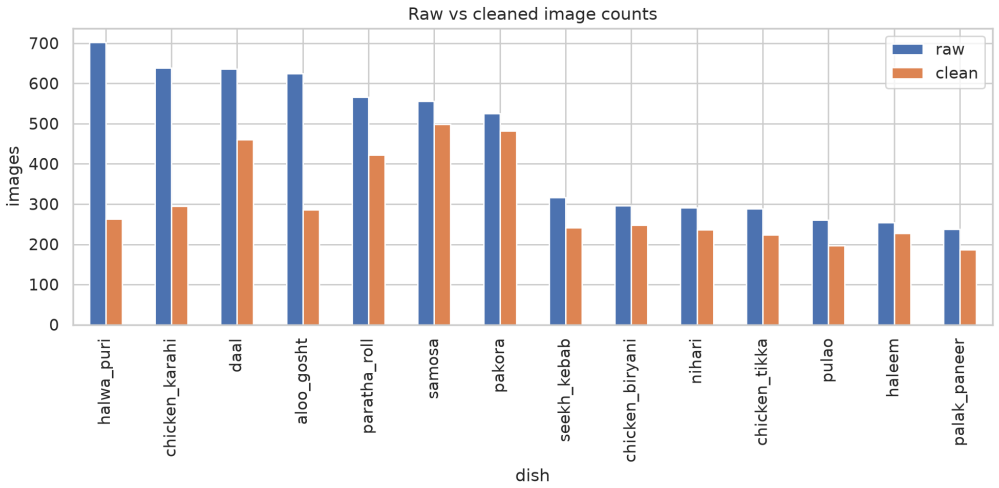


Figure 2: Raw versus cleaned image counts.

Cleaning reduced the collection from 6,193 to 4,267 images, and every dish lost some images. The largest shares were lost by halwa_puri (439 of 702, about 63%), aloo_gosht (338 of 624, about 54%) and chicken_karahi (344 of 639, about 54%), which are the three dishes that received top-up scraping and therefore contained many repeated downloads. Pakora lost the smallest share (43 of 525, about 8%).

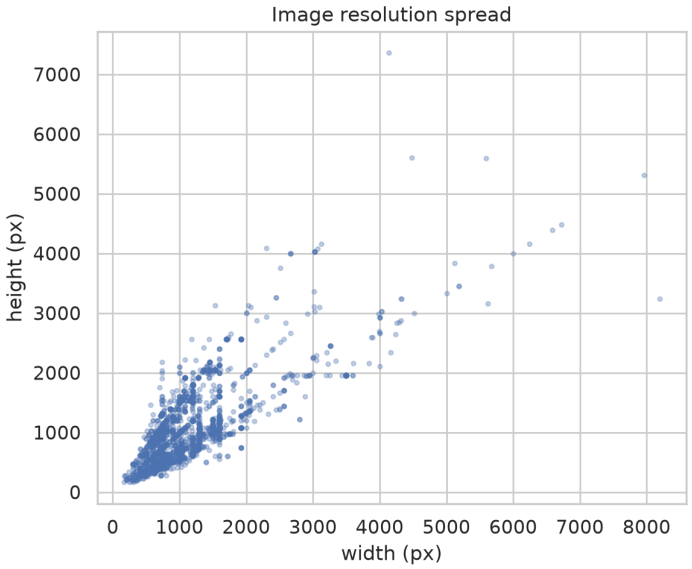


Figure 3: Image resolution spread.

Image resolution varies widely, from small images of a few hundred pixels to a few very large ones more than 4,000 pixels wide, and width and height grow together. This spread motivates resizing everything to a common 224×224 input.

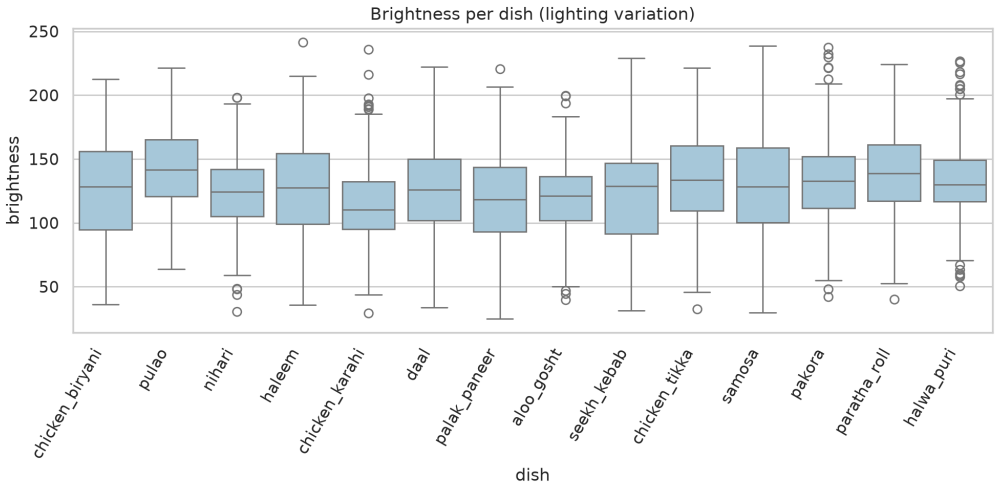


Figure 4: Brightness per dish (lighting variation).

Brightness varies widely within every dish, so each dish contains both dark and bright photos, and lighting differs between individual images more than it differs between dishes.

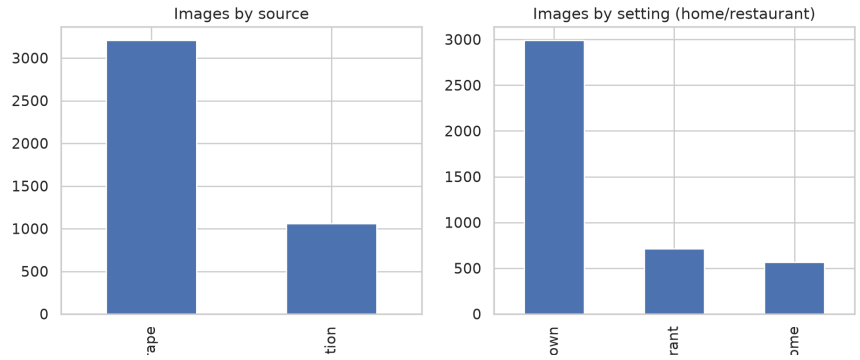


Figure 5: Images by source and by setting (home/restaurant).

Most of the cleaned images come from Bing, and a smaller share from the Kaggle dataset. The setting is unknown for most images, because only some of the search queries mention a home or restaurant setting and the Kaggle images carry no setting label.

aloo_gosht






chicken_biryani






chicken_karahi






chicken_tikka







daal







haleem








halwa_puri






nihari







pakora







palak_paneer






The sample grid shows four random cleaned images per dish. The images vary widely in angle, plating and background. The grid also exposes the problems listed under the limitations, such as watermarks, text overlays and occasional images that do not show the labelled dish.

2.5 Limitations

- **Watermarks and text overlays:** images scraped from Bing can contain watermarks, captions or text overlays, which the automatic blur and duplicate filters do not remove. Stock-photo watermarks (for example Alamy and Shutterstock), text banners (many in Urdu), video-thumbnail graphics with a "subscribe" button or a presenter's face, and even a menu page were found in the cleaned images. In a random sample of 24 images per dish, about half of the aloo_gosht and halwa_puri images (the dishes with the most top-up images) and about one in five chicken_karahi images were affected. Some banners even repeat the dish name, so a model could learn to read the text instead of recognising the food.

- **Label noise:** a few cleaned images do not appear to show the labelled dish or are not photographs of food (for example a cartoon illustration and a plate of chaat among the halwa_puri images, a restaurant menu page among the chicken_karahi images, and off-topic images in palak_paneer). The cleaning filters cannot detect these and the dataset was reviewed only by sampling, so some mislabelled images probably remain.

- **Source bias:** four dishes (samosa, pakora, daal and paratha_roll) draw on a single Kaggle dataset, while the other ten come only from Bing. The two sources may differ in photographic style, so a model could partly learn the source instead of the dish.

- **Close substitutes:** the Kaggle classes `dal_makhani` and `kaathi_rolls` stand in for daal and paratha_roll. They are similar dishes but not identical to what the project is meant to recognise.

- **Class imbalance:** raw counts range from 237 to 702 images per dish and cleaned counts from 187 to 498. All 14 dishes are above our 150-image target, but only after the top-up scraping (chicken_karahi was at 128 after the first cleaning). The smallest class, palak_paneer, has 187 images, so with a 15% split its validation and test sets hold only about 28 images each and its per-class results will be noisy.

- **Repeated downloads:** the top-up queries returned many images that were already collected, so a large part of the new downloads was removed as duplicates. The three top-up dishes therefore keep a smaller share of their raw images than the others. Some near-duplicates still survive: in the aloo_gosht sample, what appears to be the same photo occurs twice with different crops, which the perceptual-hash threshold did not catch.

- **Licensing:** the scraped images belong to their original owners. They were collected for academic use only and are not redistributed: the dataset is excluded from the repository.

3 Phase 3: Model Architecture and Implementation

3.1 Framework

We implemented both models in **PyTorch** with **torchvision**. PyTorch is the usual choice for custom training loops (frozen then unfrozen stages, early stopping, cosine schedules) and torchvision already ships EfficientNet-B0 with ImageNet weights. The same scripts run on a Colab T4 GPU and on CPU. Scikit-learn is left for Phase 4 metrics, not for the networks themselves.

3.2 EfficientNet-B0 (transfer learning)

The main model is torchvision EfficientNet-B0 pretrained on ImageNet (`IMAGENET1K_V1`). The original classifier is replaced by Dropout then a linear layer to 14 logits. Input tensors are $3 \times 224 \times 224$, matching the Phase 2 JPEGs. After the backbone, EfficientNet applies global average pooling to a 1280-dimensional vector, then our head.

Training has two phases, implemented in `src/train.py`:

1. **Head (5 epochs).** The backbone is frozen. Only Dropout and the linear layer are trained, at learning rate 1×10^{-3}.
2. **Fine-tune.** The last 3 entries of `model.features` are unfrozen. Training continues at 1×10^{-4} with a fresh AdamW optimiser and cosine scheduler, starting from the best head checkpoint.

Two stages keep the pretrained filters from being destroyed in the first epochs, then allow the last blocks to adapt to desi food photos.

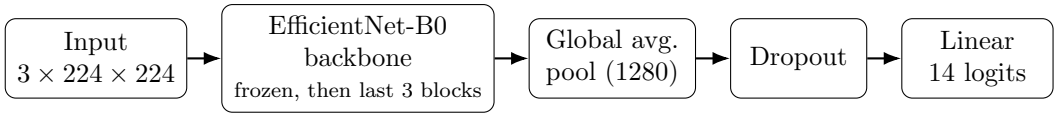


Figure 7: EfficientNet-B0 transfer pipeline used in `src/efficientnet_model.py`.

3.3 Baseline CNN

`src/baseline_cnn.py` is Abdullah's from-scratch network. The shared contract is the same I/O as EfficientNet: class `BaselineCNN(num_classes=14)`, input $3 \times 224 \times 224$, output 14 logits, trained with `python src/train.py --model baseline`.

TODO: file not in the repository yet. Phase 3 cannot list its layers. When it lands, describe it here at block level (conv / pool / fully connected) without repeating Phase 4 metrics.

3.4 Loss, optimiser, schedule, regularisation

- **Loss.** Cross-entropy with label smoothing 0.1. Smoothing stops the head from becoming over-confident on noisy Bing labels (Phase 2 noted watermarks and a little label noise).
- **Optimiser.** AdamW on *trainable* parameters only, with weight decay 1×10^{-4}. AdamW is the usual default for transformer and CNN fine-tuning; excluding frozen weights avoids useless states in the optimiser.
- **Scheduler.** Cosine annealing over the epochs of each phase. The learning rate falls smoothly and does not need an extra validation rule (unlike ReduceLROnPlateau).
- **Early stopping.** Training watches validation loss with patience 5 and writes the best weights under `checkpoints/` as `efficientnet_best.pt`.
- **Augmentation (train only).** RandomResizedCrop 224 (scale 0.8–1.0), horizontal flip, rotation of about 15°, and ColorJitter (brightness and contrast 0.2). Validation and test only resize to 224×224.
- **Normalisation.** ImageNet mean $(0.485, 0.456, 0.406)$ and std $(0.229, 0.224, 0.225)$, as named constants in `src/dataset.py`. Phase 2 saved train-set statistics in `norm_stats.json` and said they "will be used". We do *not* use those stats, because the EfficientNet weights were pretrained with ImageNet normalisation; mixing the two scales would shift every activation in the frozen backbone.

3.5 Hyperparameter search

`src/tune.py` changes *one* factor at a time around a default (head lr 1×10^{-3}, fine-tune lr 1×10^{-4}, batch 32, dropout 0.3, weight decay 1×10^{-4}, label smoothing 0.1). Each trial ran at most 8 epochs on a Colab T4, without writing a test-pred file. Table 6 is copied from `results/hparam_results.csv`. The best row (lowest validation loss) is in bold.

Run	lr	Batch	Drop.	Val. loss	Val. acc.
default	0.001	32	0.3	1.190	0.777
lr_3e-4	0.0003	32	0.3	1.194	0.773
lr_1e-4	0.0001	32	0.3	1.238	0.767
batch_16	**0.001**	**16**	**0.3**	**1.167**	**0.792**
batch_64	0.001	64	0.3	1.217	0.769
dropout_0.2	0.001	32	0.2	1.187	0.781
dropout_0.5	0.001	32	0.5	1.200	0.775
wd_1e-2	0.001	32	0.3	1.190	0.777

Table 6: One-factor EfficientNet search (`results/hparam_results.csv`). Fine-tune lr 1×10^{-4}, weight decay 1×10^{-4} except `wd_1e-2` (1×10^{-2}), label smoothing 0.1 throughout. Accuracies are the CSV `best_val_acc` values.

Lowering the head learning rate or raising the batch size made validation loss worse than the default. Dropout 0.2 was slightly better than 0.3 on accuracy but not on loss. Weight decay 1×10^{-2} matched the default loss. **Batch size 16** was the only change that clearly reduced validation loss (1.167 vs 1.190), so the final EfficientNet run used that batch size and otherwise the default knobs: 20 epochs, 5 head epochs, last 3 blocks unfrozen, dropout 0.3, AdamW weight decay 1×10^{-4}, label smoothing 0.1.

From `results/history_efficientnet.csv`, that full run reached its best validation loss at epoch 19 (loss 1.069, accuracy 0.830) during fine-tuning. Test-set Precision, Recall, F1 and AUC are Phase 4 (`evaluate.py`); they are not computed in `train.py`.

3.6 Training curves

Figure 8 is `results/curves_efficientnet.png`. Loss and accuracy are plotted for train and validation. The dashed line marks the first fine-tune epoch (epoch 6). Both curves improve after that line, which is the point of unfreezing the last blocks.

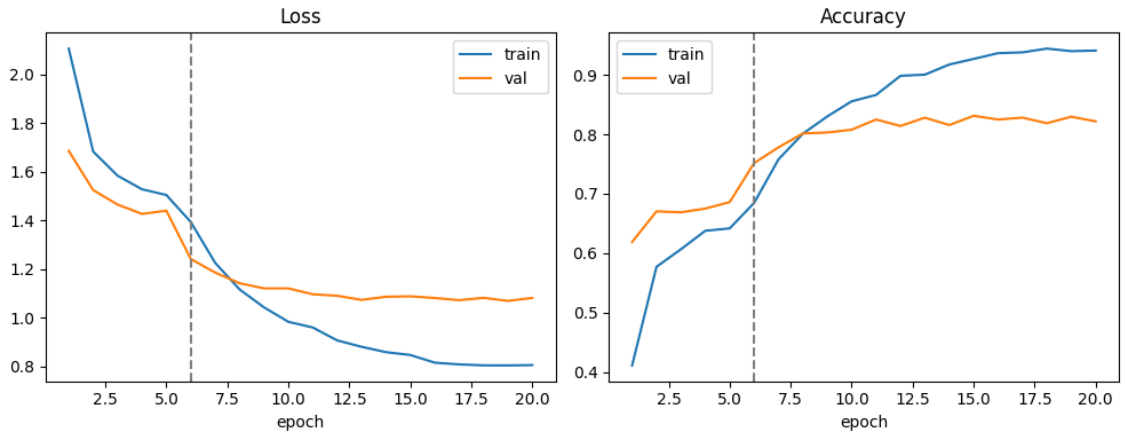


Figure 8: EfficientNet-B0 train/val loss and accuracy (`results/curves_efficientnet.png`, final 20-epoch run, batch 16).

TODO: include `curves_baseline.png` here (or in Phase 4) when Abdullah's baseline training has written it.

4 Contribution Table

Git author names below are taken from `git log`. Modules still missing from the repository are listed because they are assigned, not because they are already committed.

Member	Git author	Modules
Muhammad Huzaifa (24L-3019)	Muhammad Huzaifa	Data collection scripts (Phase 1); Phase 1 report; `src/dataset.py`; `src/efficientnet_model.py`; `src/train.py`; `src/tune.py`; `docs/COLAB.md`; Phase 3 report; README.
Muhammad Abdullah Rasheed (24L-3002)	Muhammad Abdullah Rasheed	Cleaning, preprocessing and EDA scripts; Phase 2 report; `src/baseline_cnn.py`; `src/evaluate.py`; `src/bias_check.py`; Phase 4 report.

Table 7: Who implemented which modules. Git history is the source of truth for code already in the repository.